TriageNorth: product and solution proposal

Hackathon submission design · 20 October 2026 · Working product name TriageNorth

Problem statement

Infrastructure teams receive symptoms from many monitoring systems, but resolving an incident still requires a person to assemble the surrounding facts. A container restarts, a host stops responding, and an application raises errors. These may describe one failure, yet arrive as separate alerts with different names and identifiers. Meanwhile, containers move, addresses change, and the failed instance may disappear before anyone investigates.

The operator must identify the affected resource, distinguish repeat notifications from new problems, reconstruct dependencies, retrieve logs and system state, and decide who may act. Evidence, proposed commands, approvals, and the result often live in separate tools. External ticket systems can drift from the incident, and opaque filtering makes it hard to see why a signal was discarded or grouped. Repeating that work slows response and makes automation difficult to trust.

TriageNorth turns infrastructure signals into a durable incident with an identifiable asset, an explanation supported by captured evidence, and a controlled next action. It gives operators a consistent workspace across a data center, cloud resources, hosts, and containers. Every change remains attributable to an authorized person or explicitly delegated automation principal.

The hackathon will demonstrate the complete path from a controlled failure to verified recovery. Success is a reproducible incident record and execution receipt, with measurable noise reduction and visible user control. No performance savings are claimed before measurement.

Proposed solution

TriageNorth is the submitted product: a secure AI analysis and response hub with its own landing page, navigation, inventory, topology, incident workspace, automation policies and deliverables. Event-to-Action is its first workflow. The reusable analysis model is intended to extend to recurring problems, change risk and predictive capacity warnings. The product owns operational interpretation, evidence selection and the experience of reaching a controlled next action.

OSHAL is the existing platform foundation, installed as a separately versioned Docker/OCI container baseline on Kubernetes. It supplies identity, granular authorization, workflow execution, UX services, standard connections, canonical tickets, tool admission and persistent workspaces. These inherited services are dependencies rather than hackathon inventions. TriageNorth ships as the existing `intelligent-operations` package; technical IDs and APIs stay stable while the submission uses the product name.

The signed-in person's authority and connection grants govern access. Investigators receive preprocessed evidence packets instead of unrestricted cluster access. The product determines which source and asset a signal represents, why it is filtered or grouped, what evidence is needed, and how findings and proposed work are presented. The platform supplies the shared authority and execution boundaries. One product incident links to one canonical ticket.

The deployment contract is baseline container image and storage → administrator setup → trusted public product package → scoped connection and source activation. Pin the foundation image digest and product revision separately and expose both in diagnostics. Installing the foundation makes its services available; it does not automatically activate every connector, grant every permission or establish feature parity in this product.

Market case and product economics

The target buyer operates a mixed estate and wants to preserve existing monitoring and service-desk investments. Google's SRE guidance describes alert-driven toil and the additional evidence needed to move from an impact signal to diagnosis. TriageNorth addresses that investigation and response gap with source-preserving analysis, bounded evidence and inspectable control. See [the research and source register](MARKET-RESEARCH.md).

PagerDuty, Datadog, Dynatrace, incident.io, ServiceNow and BigPanda already offer overlapping capabilities. The research compares current official pricing, distinguishes seat, infrastructure and AI usage charges, and leaves custom-quote products unpriced. Differentiation is a hypothesis to demonstrate through deployment fit, integration effort, evidence quality, user control and cost per completed investigation. No unsupported claim of unique AI correlation, automatic cost savings or market size is made.

A four-week pilot measures investigation time, actionable-alert precision, missed incidents, recommendation acceptance and full operating cost. Predictive use cases require a forecast horizon, held-out temporal evaluation, warning lead time and false-alert measurements against simple baselines. Current scheduled processing has minute-scale cadence; latency targets must be measured by stage. Production prediction, automatic evidence consumers, generated remediation and external ticket synchronization remain explicit gates in IMPLEMENTATION-STATUS.md.

The commercial offer centers on TriageNorth and the customer outcome. The existing platform is separately installed and credited in the dependency inventory. The project documents AGPL-3.0-or-later with a paid commercial-license alternative; the source reference and native packaging discrepancy are recorded in MARKET-RESEARCH.md. Platform commercial arrangements are managed separately; this proposal defines the application, its delivery requirements and measurable value. Docker/OCI packaging provides a reproducible foundation beneath the independently versioned application package.

Data center and cloud model

The inventory spans physical, virtual, and application layers. A resource can exist without a health observation. A relationship can describe configured placement without proving traffic flows across it.

Layer	Resources and relationships to represent
Facility	Site, room, rack, chassis, power supply, PDU, UPS, cooling and environmental sensor. Physical containment and power dependencies are distinct edges.
Network	External connection, router, firewall, switch, port, VLAN, subnet, virtual network, route, gateway, load balancer and network interface. Configured links, permitted paths and observed traffic remain distinct.
Compute	Physical server, management controller, hypervisor, VM, cloud instance and operating system. A VM runs on a host; a host belongs to a site or provider location.

Layer	Resources and relationships to represent
Storage	Array, pool, disk, volume, attachment and mount. An attachment is not evidence of a healthy filesystem or successful application access.
Containers	Cluster, node, namespace, controller/workload, pod, container instance, image digest, service, endpoint and ingress. Preserve ownership and placement changes.
Services	Application, API, database, queue and business service, with observed or declared dependencies and ownership.

An inventory record includes canonical asset ID, asset class, provider/native ID, tenant and environment, location, display name, authorized source, first and last observation times, lifecycle state, and version. Each relationship records its type, direction, source, validity interval, and whether it is configured, observed, or inferred. The map labels inferred relationships and exposes their supporting evidence.

Instances and workloads have separate identities. A replacement pod has a new instance ID; a continuing workload incident may retain its incident identity. IP addresses, mutable names, and random pod suffixes must not silently become durable asset identities. Alias mappings have time validity so a reused IP does not attach historical alerts to its new owner.

Standards and discovery

Use CloudEvents 1.0 structured JSON for normalized event envelopes and OTLP for telemetry. Define a versioned Operations identity profile using the relevant OpenTelemetry resource attributes. These conventions provide a vocabulary; they are not a complete universal asset inventory standard. The application owns the common inventory and relationship schema.

Use provider-native IDs and discovery APIs. GCP discovery uses Cloud Asset Inventory with change feeds where available; Azure uses Resource Graph supplemented by service APIs; AWS uses resource APIs and AWS Config where enabled. Coverage, API permissions, pagination, regional scope, freshness and provider-specific prerequisites remain visible per scanner. Cloud inventory permission does not automatically provide guest OS, container, or network traffic visibility.

Kubernetes discovery uses the API's list/watch pattern plus configurable periodic full reconciliation. Discover controllers, pods, containers, services, endpoints and nodes; relate nodes to cloud or physical hosts through verified IDs. Restart watches safely and relist after an expired resource version. Standalone container hosts use a separately authorized collector with runtime inventory permissions. Do not expose a privileged container-runtime socket to the web application.

For physical systems, support Redfish inventory where available. Align network topology interchange with the network, node, link and termination-point concepts of RFC 8345; add domain-specific attributes explicitly. Network and hypervisor discovery are separate connectors with declared capabilities. Facilities and undocumented dependencies can begin as authorized imports or manual declarations, visibly identified as such.

Every scanner supports enable/disable, scope, interval, timeout, retry/backoff, concurrency and manual refresh. Continuous updates complement scheduled scans. A failed, unauthorized or incomplete scan marks coverage stale or partial; it does not delete unseen resources. Only a successfully completed authoritative reconciliation may retire resources absent from its own scope. Preserve deletion tombstones and historical placement for incident evidence. Rate limits and destructive inventory changes require

bounded reconciliation guards.

Inbound identity and administration

Every named source has a stable source ID, owner/scope, authenticated connection, allowed event types, schema version, adapter version and inventory authority. Authenticate before accepting data and ensure the payload's source agrees with that connection. Record raw accepted content durably before acknowledging receipt. Delivery retries are idempotent on the authenticated source and event ID. A changed payload reusing the same ID is a visible conflict.

CloudEvents event intake requires id, source, specversion and type. The application profile additionally requires event time, data schema, environment, resource class and sufficient scoped native identifiers to resolve an asset. OTLP retains its native encoding and batch semantics; a telemetry adapter validates the equivalent resource identity profile without wrapping every signal in CloudEvents. Reject malformed or unauthorized deliveries with a clear contract response; persist a bounded redacted diagnostic when permitted. Valid deliveries with unresolved or ambiguous asset identity enter an explicit unmapped queue and cannot trigger infrastructure changes.

The source mapping screen maps inbound field paths and aliases to canonical resource identifiers, severity, alert type, timestamps and lifecycle state. It includes a sample payload editor, validation result, proposed asset match, conflict list and dedup-key preview. Mapping rules are declarative, typed and versioned, with preview and audit history. No arbitrary mapping code executes from this screen. Reprocessing retains both the original interpretation and the new decision, with their mapping and rule versions.

Event processing and evidence

The event flow is source authentication, durable landing, normalization and asset resolution, rule evaluation, incident consolidation and graph correlation, conditional evidence capture, ticket dispatch and controlled action. Store telemetry independently from raw deliveries and normalized alerts. Retain resource attributes, event and observation times, units, and trace/span context where applicable.

Percolation matches candidate documents against saved rules. Windowed conditions, rates, missing signals and aggregate thresholds use a separate scheduled evaluator over telemetry. Both emit the same normalized alert contract with rule version and supporting record references. Source integration, detection and remediation remain independently configurable.

Noise reduction uses stable scoped identity, including source system, canonical asset or workload, and alert type. Preserve the established key semantics, suppression windows, reserve-before-dispatch behavior, and release-on-failure behavior. Deduplicate before costly investigation and repeated evidence collection. Repeated occurrences still update counts and last-seen time. Cross-source correlation groups related symptoms but does not silently change their original identity. A resolving symptom cannot close a different alert merely because it shares a host.

Correlation evaluates incoming alerts against open incidents and the relevant topology at event time. Bound traversal depth, edge types and hub transit so a shared router or cluster controller does not merge the whole estate. Record membership when attaching a symptom. If topology is unavailable, use an explicit fallback and disclose degraded correlation.

Each matching rule specifies evidence policy: none, capture on first occurrence, capture on meaningful change/escalation, or explicit operator request. Apply byte, time, rate and cost limits. Capture volatile

facts promptly before a container disappears; retain source-captured snapshots when the source can preserve them earlier. A historical query is labeled as such and is not presented as a contemporaneous snapshot.

Anomaly detection is a separate service over historical telemetry, with detector versions, baseline readiness, per-resource windows, scores and confidence. It emits alert candidates into the same filtering and noise-reduction pipeline. Cold start, missing samples, late arrivals and failed detectors remain visible. A Prometheus threshold, imported anomaly result or score adjustment does not establish that this service is running. Detector calibration and false-positive/latency acceptance are required.

The information packet contains the originating alerts, inventory identity, event-time topology, available logs/metrics/traces, host and container state, relevant changes, external evidence, and the collection manifest. Optional Splunk or other external queries run through explicitly authorized connections. Each item records source, query/collector version, target, capture time, event-time coverage, content hash, redaction, truncation, and collection status. Timeout, denied access and missing history are visible. Failed optional enrichment does not erase the incident or imply the evidence is complete.

Application experience

Use an application-owned shell with a persistent environment selector, search, freshness indicator and current access summary. The left navigation contains Overview, Topology, Assets, Data and Decisions, Incidents, Ticket Synchronization, Approvals, Automation, Deliverables, and Administration. Show the assistant in incident context as a secondary panel; the primary task stays visible without using chat.

Screen	Main layout and user decision
Overview	Coverage and freshness strip; active incidents ranked by impact; topology preview; approval inbox; recent verified outcomes. Source outages and unknown health remain visible.
Topology	Central graph with site/cloud/cluster grouping; filters for environment, layer, source and health; right inspector with asset identity, edges, observations and incidents. Separate current and incident-time views.
Data and Decisions	Separate raw-delivery and telemetry views; normalized identity, mapping/rule versions, filter outcome, delivery retry, symptom reduction, correlation membership, snapshot decision and replay lineage. Inspect reasons before investigation.
Ticket Synchronization	Canonical platform ticket with linked external records, inbound/outbound direction, current-caller connection, field ownership/status mapping, operation/revision history, conflict review and uncertain-create reconciliation.
Assets	Searchable inventory with provider/native IDs, location, owner, lifecycle, observed health and last-seen time. Details show aliases and placement history.
Incidents	Queue and incident workspace with summary, affected service, timeline, symptoms, evidence packet, proposed work, approvals and execution history.
Approvals	Requests grouped by environment and risk; exact changes and targets; evidence age; policy evaluation; approve, reject, or request revision. Approval controls depend on the viewer's authority.

Screen	Main layout and user decision
Automation	Policy list and editor for trigger, allowed action, exact scope, execution principal, limits, expiry, verification and rollback behavior. Preview explains permitted and refused actions.
Deliverables	Versioned investigation reports and work bundles, file viewer/diff, validation, downloadable package, approved execution option and receipts.
Administration	Connections and Credentials, Sources and Mappings, Discovery, Detection Rules, Access, Retention, and Audit. Keep secrets separate from routing and action policy.

The connection setup page uses a left-hand list of named connections, a central form, and a right-hand effective-access panel. The form shows provider, account/subscription/project, environment, credential reference, owner and allowed capabilities. Existing references are masked; secret values are entered only through the shared secure connection flow. Separate actions test authentication, inventory reads and evidence reads. Display what was tested and when. Write access is a separate grant and test; a successful read probe must not display an unrestricted Connected status.

The incident workspace uses three columns: an asset and impact summary, a central evidence/timeline/work area, and an approval panel. The panel explains exact targets, requested action, risk, principal, expiration and missing conditions. An operator can inspect or download a proposal without authorizing execution. Mobile stacks these sections and preserves the target and approval summary above the final action.

Data inspection and integration directions

The experience distinguishes inbound alert integrations from outbound ticket synchronization. Prometheus/Alertmanager adapters, standardized CloudEvents input and native OTLP intake use named authenticated sources. Source configuration belongs to an authorized signed-in user; source or webhook authentication still governs each background delivery. Optional Splunk enrichment is an independently authorized read. Inbound intake permission never becomes outbound write or infrastructure authority.

The data view explains the path from raw accepted input or telemetry to normalized alert, filter decision, delivery deduplication, stable symptom key, suppression/reservation, correlation, information packet and canonical ticket. Operators can inspect versions, record IDs, reason codes, counts and timestamps. An exact delivery retry is not another independent occurrence. Different alert types and sources retain their identity; cross-source correlation is an explicit membership decision. The established production reduction algorithm must be inspected and reused, including windows and failure/restart semantics. The prototype's severity and tuple demonstration does not replace it.

Telemetry is stored and inspected separately, with native resource attributes, signal kind, units, event/observation time and supporting rule/window references. Document percolation and scheduled aggregate evaluation remain distinct. Show invalid, unauthorized, unmapped, unmatched, suppressed and dispatched outcomes without exposing content outside the caller's scope. Replaying input records a new interpretation and leaves the original decision inspectable.

Standard external ticket synchronization

ServiceNow, GitHub, GitLab and Jira are target external ticket integrations through the standard connection framework. One canonical platform ticket links to external records; connection selection and provider calls run under the caller's admitted identity or an explicitly activated scoped background principal. Personal connections remain personal; shared connections require native grants. The experience renders and configures this workflow without introducing another credential or ticket system.

Each synchronization policy specifies outbound-only or two-way direction, exact instance/project/repository, field ownership, status mapping, comment handling, attachments/evidence redaction and conflict handling. Retain provider/connection/account, external record ID, remote/local revisions, origin marker, last result and operation history. Prevent feedback loops and apply ordered/deduplicated verified webhook changes. External closure never proves infrastructure recovery.

Reserve a durable synchronization operation before issuing an authorized write. An internal operation key does not make a provider POST idempotent. If a create response is lost, reconcile through a proven external record/origin relationship or require manual review; never blindly create another ticket. Expired/revoked access, provider denial and revision conflicts remain visible and pause work. Preview and confirmation use the platform's shared write boundary, rather than a client-side checkbox as authority.

Source review on 20 October 2026 confirms all four connector declarations in the kernel checkout, but not complete live bidirectional synchronization. ServiceNow currently declares read-only resources; the other declarations have differing create/comment/webhook coverage. Missing update, transition, guarded write and webhook operations must be completed in the standard framework and tested per provider. Package allow-lists, installed connector admission and live-account acceptance are separate release gates.

Access and automation control

Connection ownership does not authorize every application user to use the connection. Source scope, inventory visibility, evidence access, ticket access and action authority must all pass the platform's checks. An environment administrator is not automatically a production executor. The following roles are convenient bundles of capabilities; users receive resource-scoped grants, not an all-powerful application role.

Capability	Viewer	Investigator	Approver	Automation administrator	Executor
Read authorized assets and incidents	Yes	Yes	Yes	Yes	Yes
Collect authorized evidence	No	Scoped	No by default	No by default	No by default
Generate a work proposal	No	Scoped	No by default	No by default	No by default
Approve a concrete request	No	No by default	Scoped	No by default	No by default

Capability	Viewer	Investigator	Approver	Automation administrator	Executor
Configure delegation policy	No	No	No	Scoped	No
Execute infrastructure changes	No	No	No	No	Scoped
Manage connection references	No	No	No	Separate grant	No

Separate the operational modes: Observe, Investigate, Propose, Execute once, and Execute under policy. Installation enables no unattended changes. Observation and read-only investigation may be scheduled only after an authorized owner activates them with an execution principal and limits. One-time execution requires approval of a concrete work version. Unattended execution requires an explicitly approved policy for a known action/template, with constrained parameters, resource scope, environment, expiry, concurrency, rate/cost limits and stop conditions.

An approval binds the bundle digest, action/template version, target IDs, parameters, connection reference/version, evidence and topology versions, principal, scope and expiry. Immediately before execution, recheck grants, policy, target incarnation, required preconditions and approval validity. A changed artifact, target, principal or relevant precondition requires a new decision. Permission withdrawal or expired delegation stops new work. In-flight behavior must be declared per action and audited; interruption is not assumed to undo an external mutation.

Operators can pause a policy or revoke an execution grant from the application. Production policy may require a different approver from the requester. Approval authority never increases the executor's provider permissions. Imported telemetry, logs and generated text are untrusted data; they cannot grant authority or change tool policy.

Generated code and script delivery

Investigation produces a versioned work bundle attached to the canonical ticket and indexed in Deliverables. A bundle includes report.md, evidence-manifest.json, plan.json, preflight and diagnostic scripts, a remediation script or typed action plan, verification steps, rollback instructions where feasible, validation results, README instructions and a SHA256 manifest. An optional deterministic ZIP carries the same files. Record the originating ticket, evidence references, generator/template versions, target requirements and tool dependencies. Do not embed credentials or environment secret values.

The application displays files, diffs, rationale, expected effects, validation results and rollback limits before approval. Validation includes appropriate schema/lint/static checks, secret detection, target/parameter checks and a dry run or disposable fixture when the tool supports it. Label unvalidated or non-dry-runnable operations explicitly. A downloadable bundle is a proposal, not permission to execute.

The native execution path uses registered platform tools or a supported sandboxed runner contract with explicit capabilities and target scope. A newly generated arbitrary script is downloadable and reviewable, but cannot automatically become an executable tool. It must pass the registration/review path before policy automation can call it. Do not introduce shell execution in an application route to bypass that boundary.

Each execution produces an immutable receipt containing the exact approved digest, targets and incarnation IDs, action, principal, approval/policy IDs, start/end times, redacted bounded output, exit status, postcondition results, costs and rollback outcome. Successful command exit alone does not resolve an incident. Verification must confirm the intended effect. If it fails, retain the incident, stop or retry within bounds, and escalate visibly. Rollback is attempted only when the action declares a safe authorized rollback; irreversible changes remain clearly identified before approval.

Downloads can be executed outside the platform. The application cannot enforce its gates in an external shell. Such work remains Externally executed or Outcome unverified until separately supplied evidence establishes the result; it must never inherit a native execution receipt.

Data ownership and records

Record	Purpose and important bindings
Source and mapping version	Authenticated source, scoped owner, adapter/schema and deterministic identity interpretation.
Raw delivery	Source plus event ID, immutable accepted content/reference, content hash, receipt and replay lineage.
Telemetry observation	Signal, resource identity, timestamps, units/context, source and retention class.
Asset and asset observation	Canonical identity separate from temporal provider state, aliases and lifecycle.
Relationship observation	Typed endpoints, evidence/source and validity interval; preserves event-time topology.
Detection rule and evaluation	Versioned rule, window/input references, matched outcome and reason.
Alert and incident membership	Symptom identity, occurrence counts, incident attachment and consolidation decision.
Incident	Operational identity and lifecycle linked to one canonical platform ticket; no duplicate ticket engine.
Evidence packet and items	Capture policy, bounded content references, collection receipts and completeness.
External ticket link and sync policy	Canonical ticket, scoped connector/account/target, external record, field ownership, direction and status mappings.
Sync operation and receipt	Reserved operation identity, local/remote revision, origin, confirmed write, outcome, retry/reconciliation and webhook lineage.
Work bundle and validation	Immutable version/digest, files, target requirements and validation evidence.
Approval or delegation policy	Specific authority, artifact/action bindings, targets, limits and expiry.
Execution and verification receipt	Exact action and identity, effects, postconditions and terminal result.

Scope every record and content lookup using the canonical platform authorization context. Incident membership, graph traversal, search, exports and assistant retrieval must enforce the same data boundary. Retention settings distinguish high-volume telemetry from incident evidence, approval and

execution history. Show legal holds and referenced evidence before purging; keep content hashes and deletion receipts where policy permits.

Reliability and documented pitfalls

Preserve raw accepted inputs before acknowledgement; retry processing from durable state. Use leases, idempotency keys and durable reservations for work that can be retried. Explain the recovery decision when an external action timed out with an unknown outcome; do not blindly repeat a potentially completed mutation.

Count every decision: accepted, invalid, unmapped, unmatched, consolidated, correlated, evidence skipped/failed, parked, dispatched and execution outcome. Monitor missing input, a stopped scanner, growing backlog and a stage producing nothing while upstream work arrives. A pipeline can fail silently while its process remains healthy.

Do not suppress after expensive analysis, deduplicate only by hostname, use mutable prose in incident keys, reconstruct membership from a later time query, or collapse all nodes through a shared infrastructure hub. Do not offer auto-resolution or rollback controls without working implementations. Bound background work and carry its explicitly activated identity through every data and tool call.

Hackathon demonstration and delivery

The complete product design covers data center, AWS, Azure, GCP and container inventory. The proposed hackathon vertical slice proves one real disposable container environment end to end, plus replayable synthetic cloud discovery fixtures for all three providers. A provider is labeled Live only after its actual connection and discovery pass acceptance. Physical facilities, broad network discovery and additional production remediations are later increments; they remain visible in the architecture as planned coverage.

Step	Demonstration and acceptance evidence
Set up	An administrator registers a source, binds its credential reference, previews identifier mappings and enables discovery for a disposable environment. Invalid identity is visibly refused or parked.
Establish context	Discover a small service, its containers/nodes and a dependency. Show inventory freshness and a topology graph backed by the same records correlation reads.
Trigger a failure	Create a controlled failing workload. Retain alert payload and capture current/previous container evidence before replacement. Label any fixture data.
Inspect decisions	Trace raw versus normalized input and separate telemetry; explain filtered, quarantined, duplicate, consolidated and snapshot outcomes with versions and reasons.
Synchronize work	Link one disposable external ticket through the caller’s standard connection; preview mapped fields, demonstrate denied/expired access and uncertain-create reconciliation. All four providers remain target coverage.
Reduce noise	Replay 100 occurrences of the same symptom. Expect one active incident/ticket, occurrence count 100 and one initial investigation; different alert types remain separate symptoms.

Step	Demonstration and acceptance evidence
Preserve movement	Replace or reschedule the container. Show the new instance, historical placement, and continuing workload incident without rewriting prior evidence.
Enrich	Attach an optional external evidence item or show its explicit unavailable state. Display the packet manifest and collection times.
Propose work	Generate a versioned report and work bundle. Inspect files, validation and exact targets before approval.
Exercise control	Refuse execution for a viewer; approve a specific version as an authorized approver. Edit the bundle and prove the old approval cannot authorize it.
Verify recovery	Execute an approved registered action in the disposable environment, verify recovery and attach the receipt. A simulated failed postcheck keeps the incident open.
Delegate narrowly	Enable a time-limited policy for the same known safe action in the disposable scope; prove pause/revocation blocks the next execution.

Measure event-to-record latency, evidence capture delay, occurrences per incident, investigations per incident, discovery freshness, approval turnaround and verified recovery time. State the environment, input workload and observed result; do not present targets or fixtures as production measurements.

Deliver as one installable application package with a manifest, application-owned UI, source and discovery adapters, schema migrations, declared shared dependencies, scoped authorization catalog, detection/evidence policies, registered tests, sample payloads, installation guide and teardown instructions. Keep implementation source separate from per-incident generated bundles. Pin package and artifact versions, record source provenance and publish through the repository's required local checks and enabled hooks. Package activation must not silently activate production discovery, credentials or remediation policies.

Suggested implementation sequence: package and access foundations; source identity and durable intake; inventory and topology; detection and consolidation; evidence packets and ticket workflow; deliverables and bound approval; registered execution and verification; then cloud coverage and operational hardening. Each increment needs an observable acceptance result before expanding scope.

Design review decisions

TriageNorth is the working product name; the technical package slug remains `intelligent-operations`. Finalize shared native approval/runner gaps, provider coverage for the hackathon and permitted demo remediation against the recorded local implementation. The experience-slice boundary, shared current-caller connections, explainable data processing and standard external-ticket synchronization are accepted design decisions in DECISIONS.md and ADRs 0001–0004, with product positioning refined by ADR0016. Open items remain the first live sync provider, provider accounts, hackathon date and exact registered remediation. They do not authorize production activation.

Standards references

- CloudEvents specification: https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md
- OpenTelemetry resource conventions: https://opentelemetry.io/docs/specs/semconv/resource/

• OpenTelemetry cloud conventions: https://opentelemetry.io/docs/specs/semconv/resource/cloud/

• Kubernetes API list and watch: https://kubernetes.io/docs/reference/using-api/api-concepts/

• GCP Cloud Asset Inventory: https://docs.cloud.google.com/asset-inventory/docs/asset-inventory-overview

• Azure Resource Graph: https://learn.microsoft.com/en-us/azure/governance/resource-graph/

• AWS Config resource relationships: https://docs.aws.amazon.com/config/latest/developerguide/config-concepts.html

• DMTF Redfish: https://www.dmtf.org/standards/redfish

• IETF network topology model RFC 8345: https://www.rfc-editor.org/rfc/rfc8345